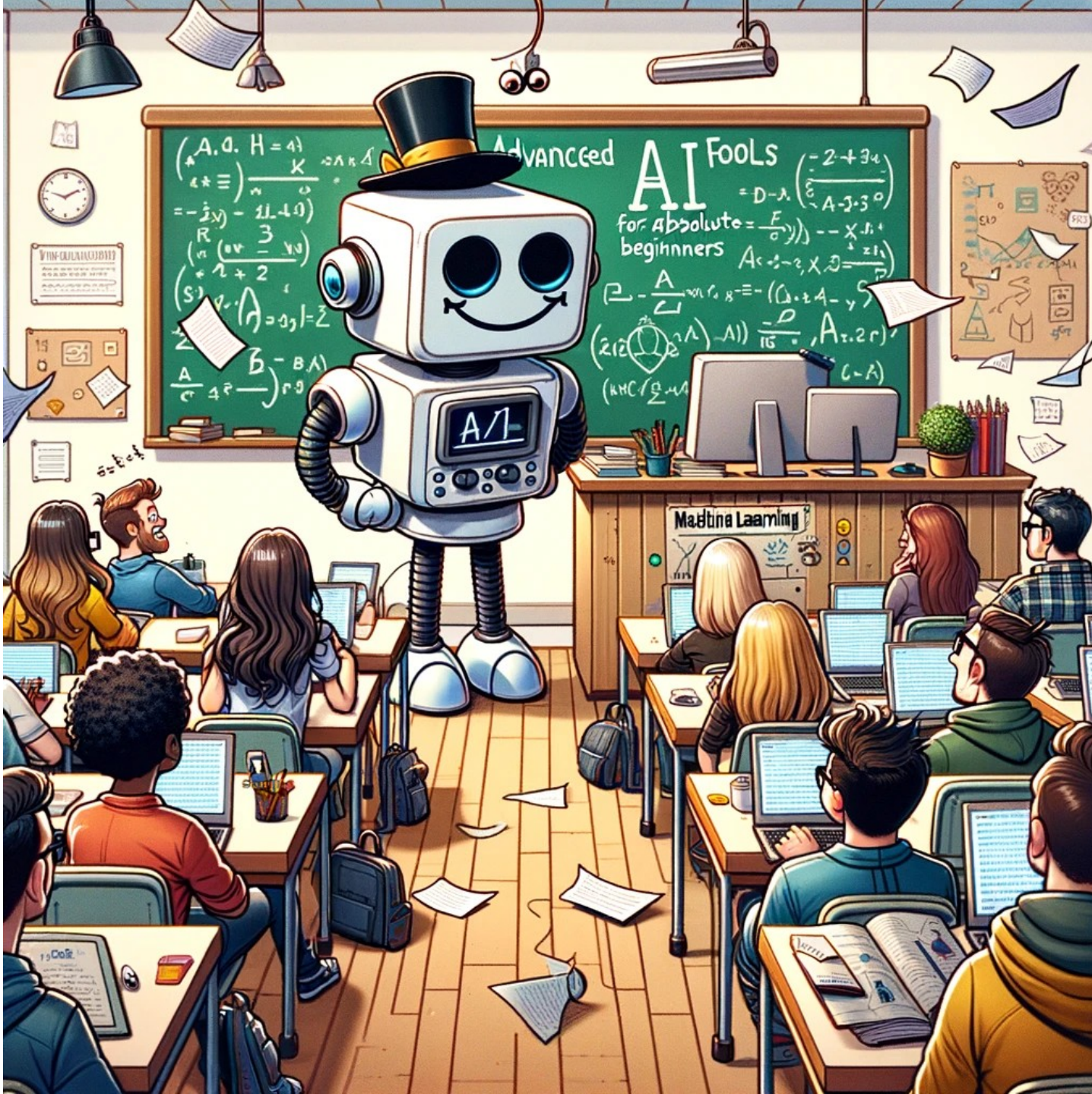




CS 229: Machine Learning

Spring quarter – March 30,
2026

Instructor: Greg {Ver Steeg

TA:

Partha Thakuria



Karpathy:
“LLM research is [about] summoning
ghosts”

Amodei (Anthropic CEO):
“Imagine a country of geniuses in a
datacenter”

Just next-token prediction!

And we are *just* a bundle of neurons

What makes it work?

- Powerful enough model
- Trained with large and complex enough data
- Optimizing right thing in right way, and
- Engineered to scale



First week or two – getting into some details

Some other topics:

NLP

- Tuning (LoRA) vs in context learning
- Scaling laws

Vision

self-supervision/contrastive

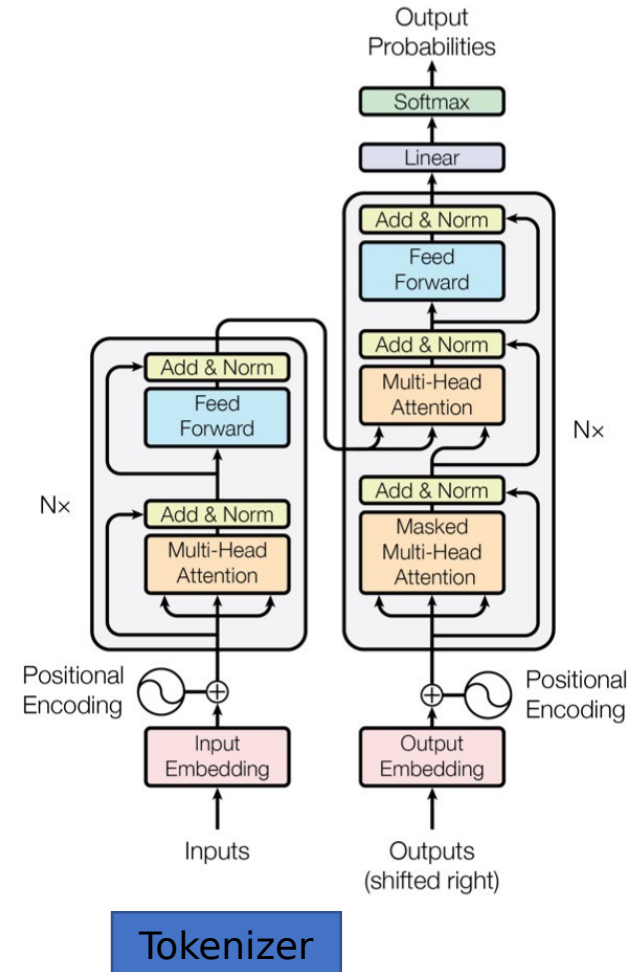
Uncertainty quantification

Deep Bayes

Optimization dynamics

Gen. models (focus on flow matching)

Graph neural nets



Course logistics

On course website

<https://elearn.ucr.edu/courses/228278>

Syllabus is there

Announcement via Canvas email list

(You should have gotten one already! If not email me to be added.)

3.5 Homeworks

Homework 0	– 10%	<i>(Discuss shortly)</i>
Homework 1	– 15%	(CalcGPT)
Homework 2	– 15%	(Uncertainty quantification)
Homework 3	– 15%	(Flow matching w/ ViT)
	55%	

HW 1-3 will have some significant but challenging Extra Credits for people who want to go deep.

[Restrictions apply](#)

Pay As You Go

\$9.99 for 100 Compute Units

\$49.99 for 500 Compute Units

You currently have 0 compute units.

Compute units expire after 90 days.
Purchase more as you need them.

- ✓ No subscription required.
Only pay for what you use.
- ✓ Faster GPUs
Upgrade to more powerful GPUs.

Recommended

Colab Pro

\$9.99 per month

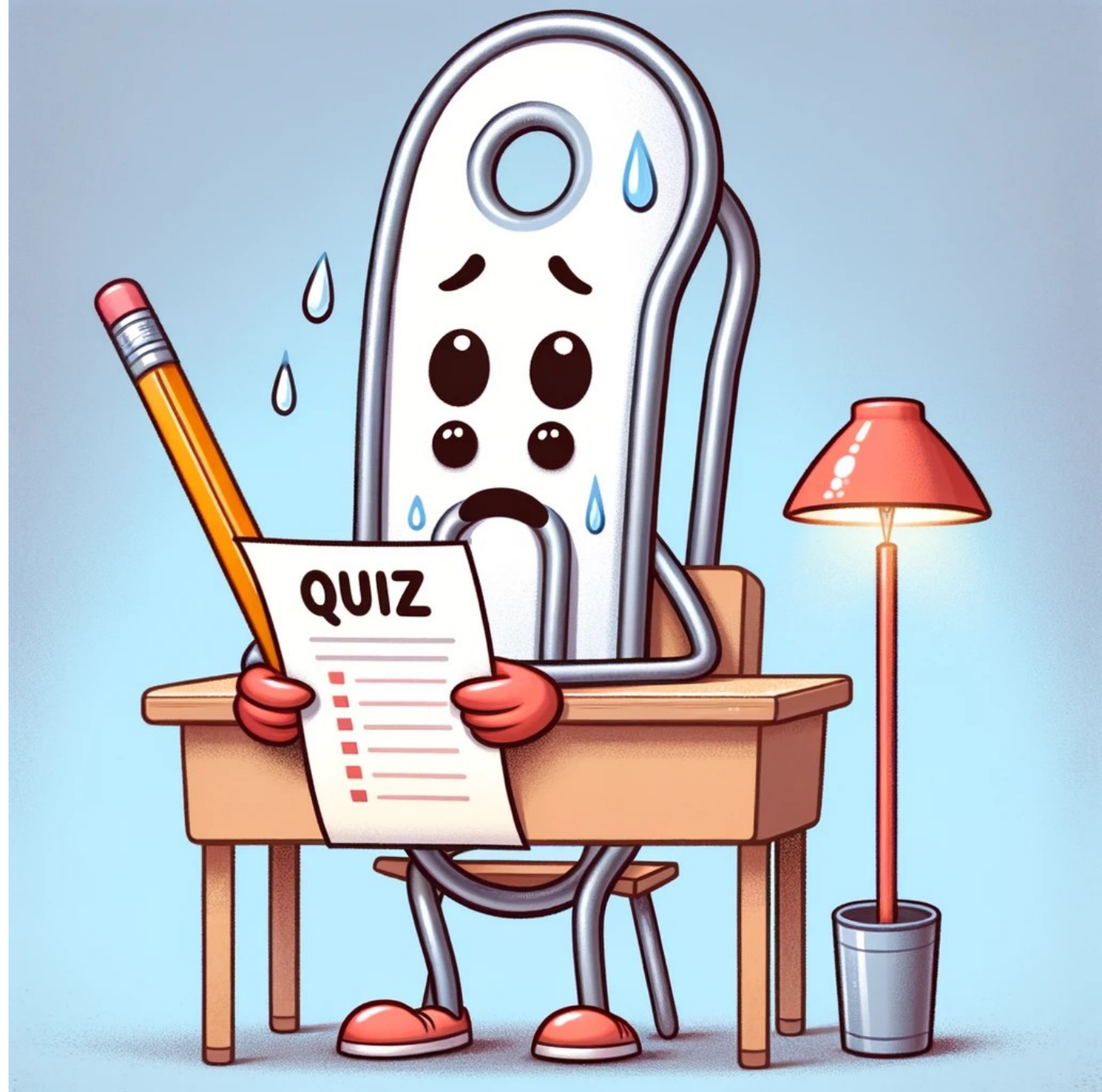
No cost for students and educators

- ✓ 100 compute units per month
Compute units expire after 90 days.
Purchase more as you need them.
- ✓ Faster GPUs
Upgrade to more powerful GPUs.
- ✓ More memory
Access our highest memory machines.

Free Colab
Pro for
students!
It has
serious
GPUs!

Quizzes – don't study (20%)

- Frequent low stakes feedback for you and me
- Do it without help – value of being wrong
- Practice for final
- New ideas
- *** Challenge problems



Final (20%)

Variations of quiz questions

GPT-4 Prompt: I need a final version of the anthropomorphized paper clip. This time he's taking a final exam and his stress levels are through the roof - ten times the anxiety of the previous pictures.



Generative AI policy / thoughts

- **Yes:** Explain
- **No:** Auto-generate

Experimental code comprehension test (5%)

Background

Quiz 0: Due Saturday. I'll overview results on Monday.
Meant to gauge background knowledge

HW 0: due April 10 (Friday, in 12 days!)

*If you feel like you are not prepared for the class,
consider auditing and letting someone on the waitlist
join.*

Canvas- Modules

Video coding demos

Below are some links to the coding demos I did in CS 224. The actual code is in the "demos" dropbox folder also linked in modules.

You may have to skip the first part of the lecture, where I usually talked about quiz results, to get to the coding part.

Note: they are in *reverse* order, so the bottom video is week 1.

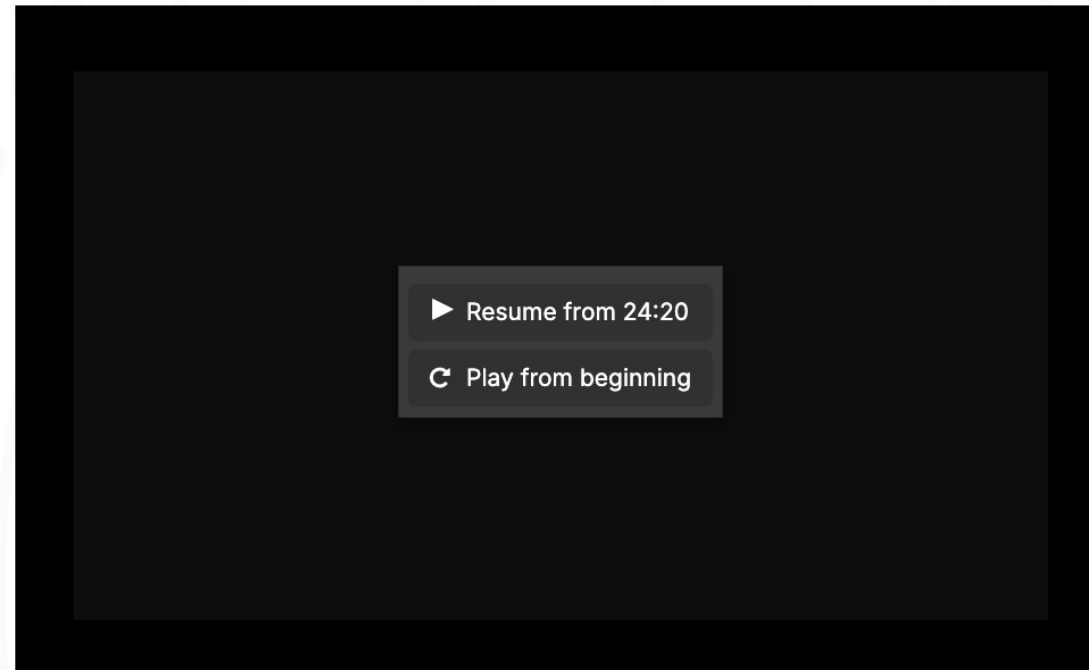
Week 1 - start at 61:35 - tensor basics in pytorch

Week 2 - start at 34:50 - models in pytorch, tokenizer

Week 3 - start at 31:20 - Cross validation (skippable)

week 4 - start at 9:31 - Multilayer perceptron, Loss, SGD (Start here if you have some experience)

week 5 - start at 17:55 - Debugging neural nets, MLP training demo (Start here if you have lots of experience and want a refresher)



Slides

Posted from 1-100 minutes before class

Link is on canvas

Books and papers

Books with good background on various topics online – we won't closely follow one

I'll email some papers to “read” that I'll cover in class



Nano banana

Office hours – see syllabus

Office hours:

1:50pm-3:00pm Monday (right after class - we can walk to my office MRB 4114 for longer chats)

6:00-7:00 pm Thursday on Zoom

TA Office hours:

10:00-11:00 am Tuesday, WCH 110

6:00-7:00 pm Friday on Zoom



HW 0: AI generated name detector

- Catch up on basic Pytorch if you are new to it
- Dive into some LLM details (tokenization, padding, batch processing) for the rest of us

“makemore” project

Social security database:
eriksen,anara,lucie...

Generated:
jieron,slan,keyanvexy...

Also check out his
“nanoGPT”
“nanoChat”
“Autoresearch”!

The screenshot displays a Jupyter Notebook interface with the following content:

build_makemore_yay Last Checkpoint: 3 minutes ago (autosaved)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

In [532]: `probs.shape`

Out[532]: `torch.Size([5, 27])`

In [544]:

```
# btw: the last 2 lines here are together called a 'softmax'

In [532]: probs.shape
Out[532]: torch.Size([5, 27])

In [544]:
nlls = torch.zeros(5)
for i in range(5):
    # i-th bigram
    x = xs[i].item() # input character index
    y = ys[i].item() # label character index
    print('-----')
    print(f'bigram example {i+1}: {itos[x]}{itos[y]} (indexes {x},{y})')
    print('input to the neural net:', x)
    print('output probabilities from the neural net:', probs[i])
    print('label (actual next character):', y)
    p = probs[i, y]
    print('probability assigned by the net to the correct character:', p.item())
    logp = torch.log(p)
    print('log likelihood:', logp.item())
    nll = -logp
    print('negative log likelihood:', nll.item())
    nlls[i] = nll

print('=====')
print('average negative log likelihood, i.e. loss =', nlls.mean().item())

bigram example 1: .e (indexes 0,5)
input to the neural net: 0
output probabilities from the neural net: tensor([0.0607, 0.0100, 0.0123, 0.0042, 0.0168, 0.0123, 0.0027, 0.0232, 0.0137, 0.0313, 0.0079, 0.0278, 0.0091, 0.0082, 0.0500, 0.2378, 0.0603, 0.0025, 0.0249, 0.0055, 0.0339, 0.0109, 0.0029, 0.0198, 0.0118, 0.1537, 0.1459])
label (actual next character): 5
probability assigned by the net to the the correct character: 0.012286253273487091
log likelihood: -4.3992743492126465
negative log likelihood: 4.3992743492126465

bigram example 2: em (indexes 5,13)
input to the neural net: 5
output probabilities from the neural net: tensor([0.0290, 0.0796, 0.0248, 0.0521, 0.1989, 0.0295, 0.0094, 0.0335, 0.0097,
```

The spelled-out intro to language modeling: building makemore

 Andrej Karpathy

 18K

 Share

Ask

 Save

↓ Download

 github.com/karpathy/makemore



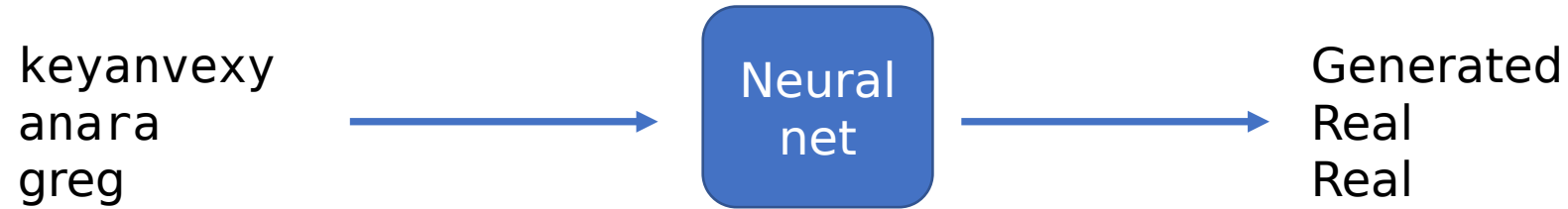
makemore

Public

Watch 38

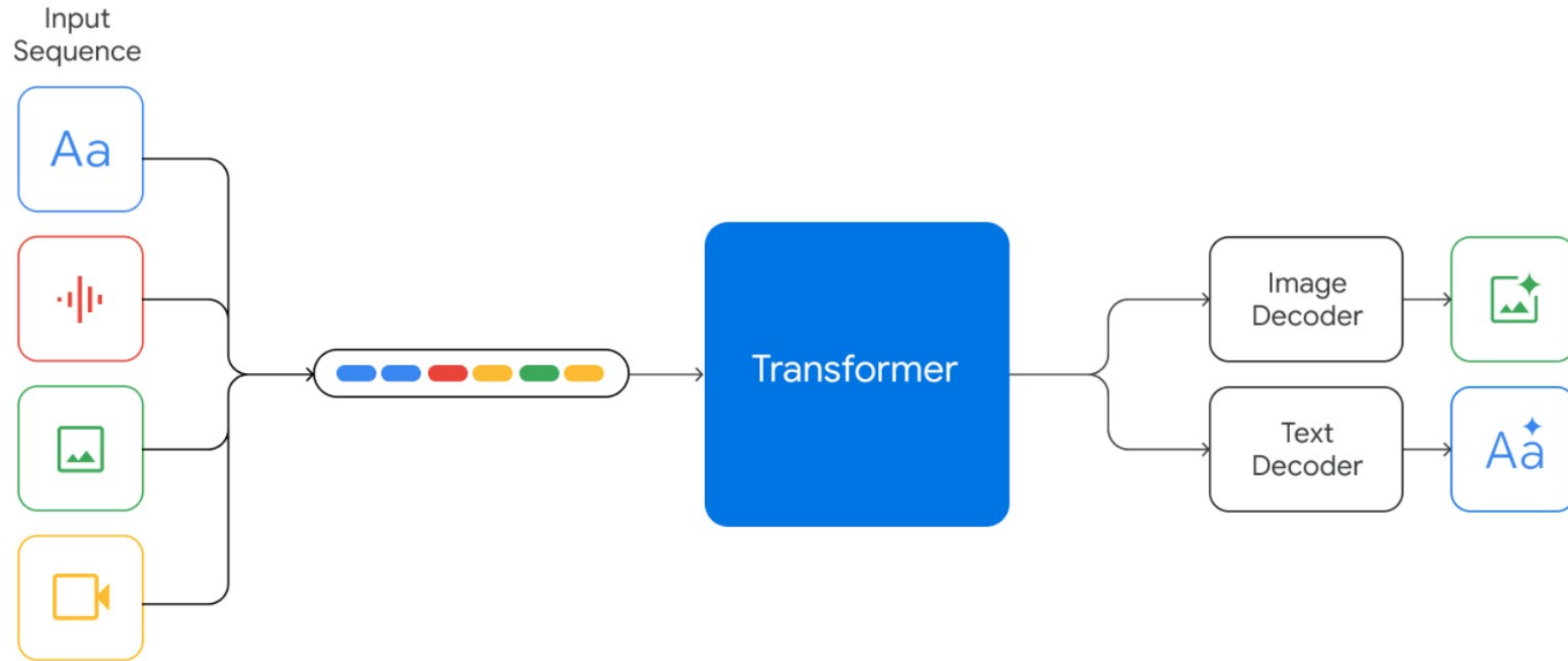
38

How to process these? jieron, slan, keyanvexy ...



General problem - turn everything into numbers, turn numbers back into everything

Multimodal LLMs



Turn text into a sequence of numbers

Keyanvexy

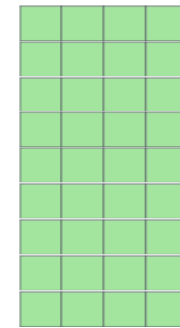
11, 5, 25, 1, 14, 22, 5, 24, 25

Then embed as *float vectors*

$E_{11}, E_5, E_{25}, \dots$

Size of the final representation?
(length of sequence, d)

d - embed dimension



Row for
each
token in
vocabula
ry

E, Embedding table

Multiple sequences in a tensor?

keyanvexy

11, 5, 25, 1, 14, 22, 5, 24, 25

greg

7, 18, 5, 7

greg

7, 18, 5, 7, 0, 0, 0, 0, 0

Pad to same length. Pad_token_id=0

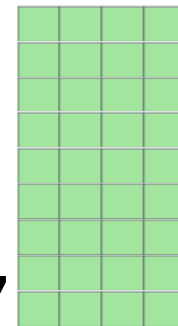
```
input_ids = torch.tensor([11, 5, 25, 1, 14, 22, 5, 24, 25],  
                          [ 7, 18, 5, 7, 0, 0, 0, 0, 0])
```

Then embed

Size of the final representation?

(Batch size, max length of sequence,

d – embed dimension



Row for
each
token in
vocabula
ry

E, Embedding table

Best way to tokenize?

Byte-Pair Encoding (BPE)

character-based
models

vocabulary size

word-based
models

f a s t e r
6 1 19 20 5 18

fast er
19731 288

faster
18277

f a s t e s t
6 1 19 20 5 19 20

fast est
19731 791

fastest
1729

q u i c k e s t
17 21 9 3 11 5 19 20

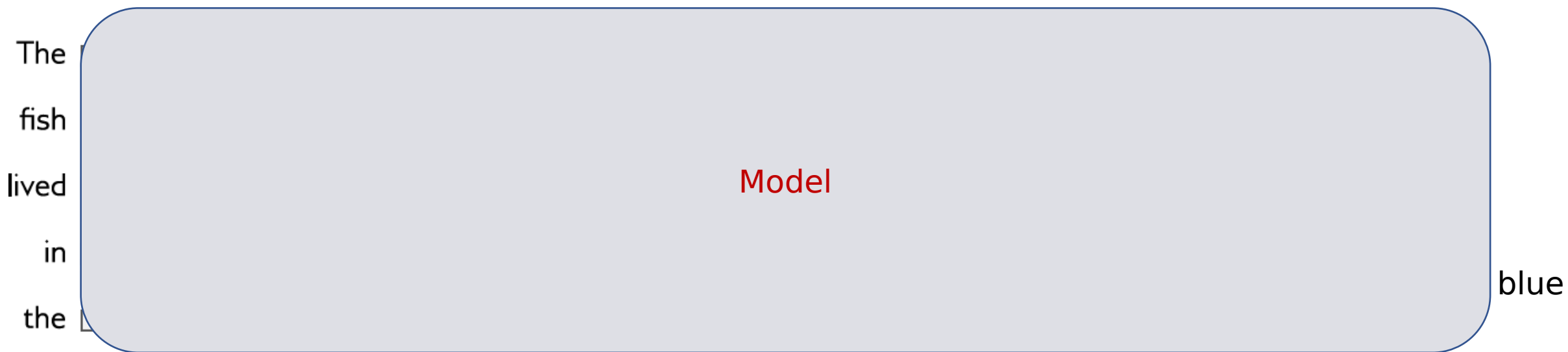
quick est
1550 791

quickest
65536

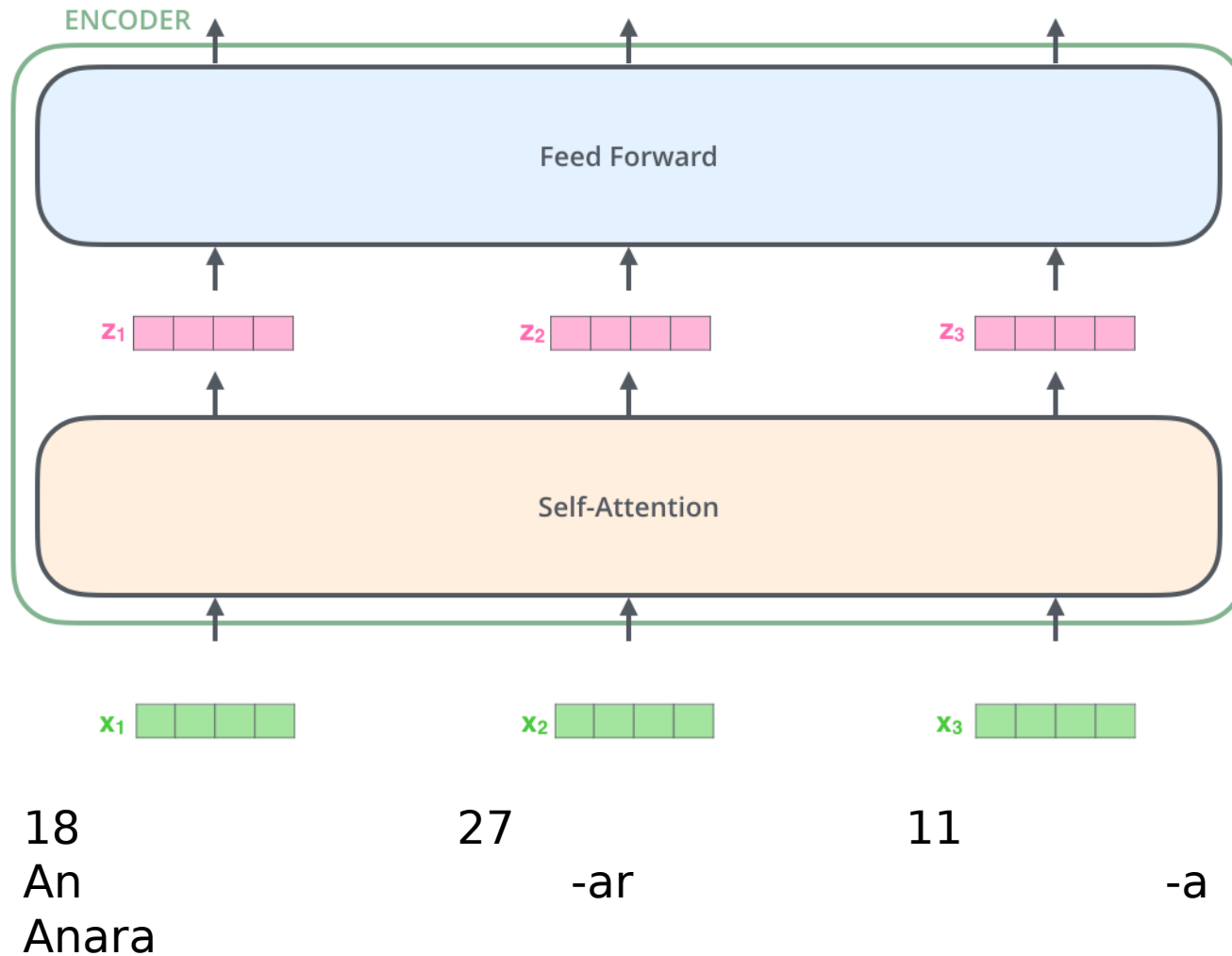
**“keyanvexy”
sequence length
is 9!**

**No
“keyanvexy”
in**

Zooming in on GPT models

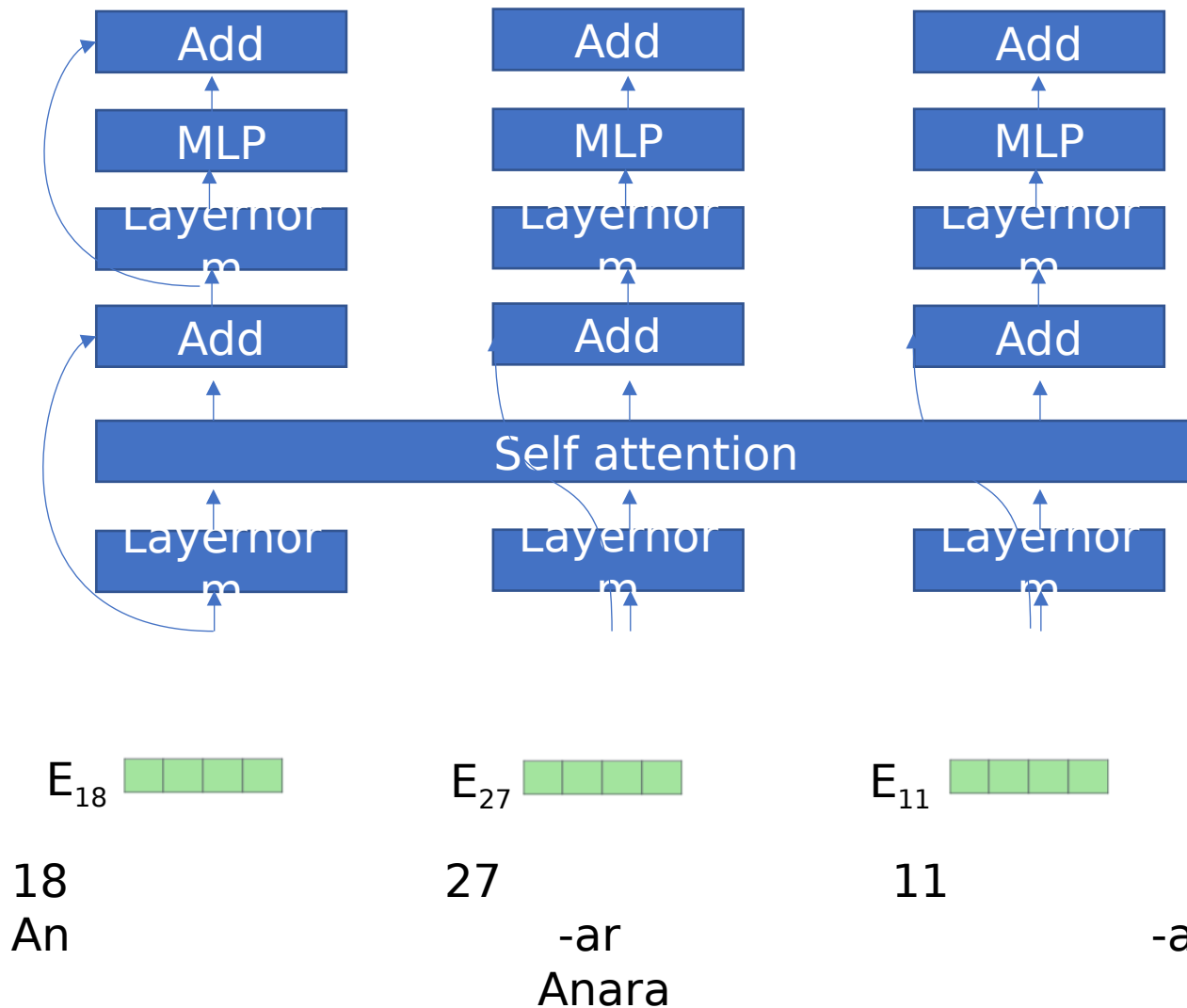


High level picture of a transformer block



- # parameters does not grow with sequence length
- Can process arbitrary length sequences, in principle
- Pad to get constant length during training

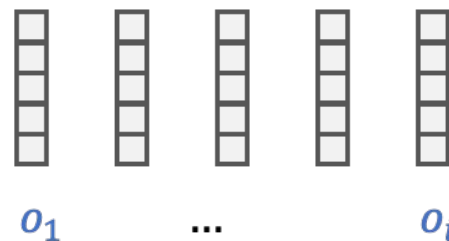
Better picture



- Most parts are processed independently, in parallel EXCEPT Self-attention
 - # parameters does not grow with sequence length
 - Can process arbitrary length sequences, in principle
 - (None of these operations know order)
- Embed dimension
- Row for each token in vocabulary
- E, Embedding table

Self-attention

First, split up embedding into three parts



Output depends on a probability distribution over the inputs

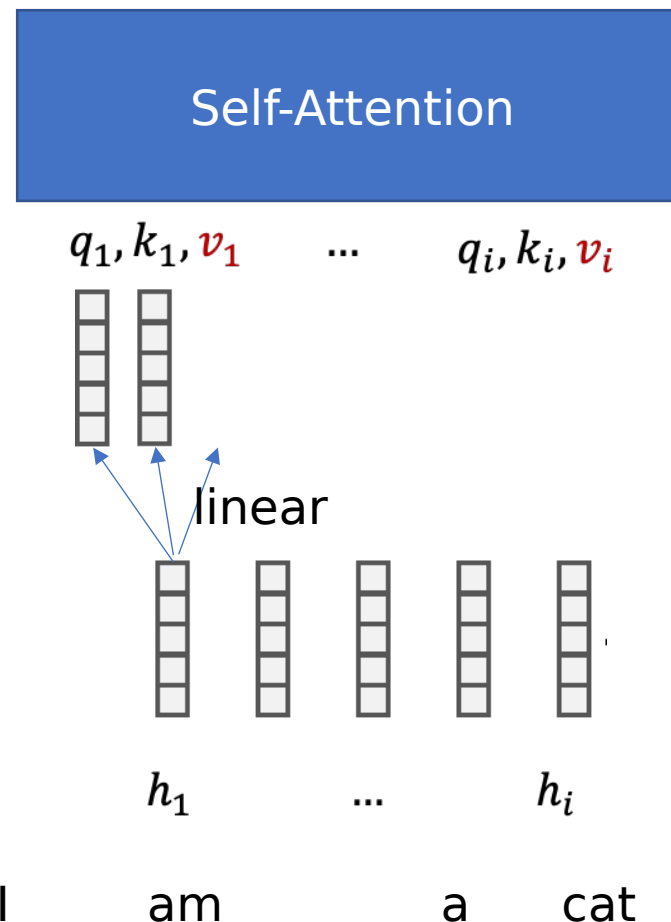
$$o_i = \sum_j p(j|i) v_j$$

How do we get the probability distribution? We always use *softmax* to get discrete probabilities

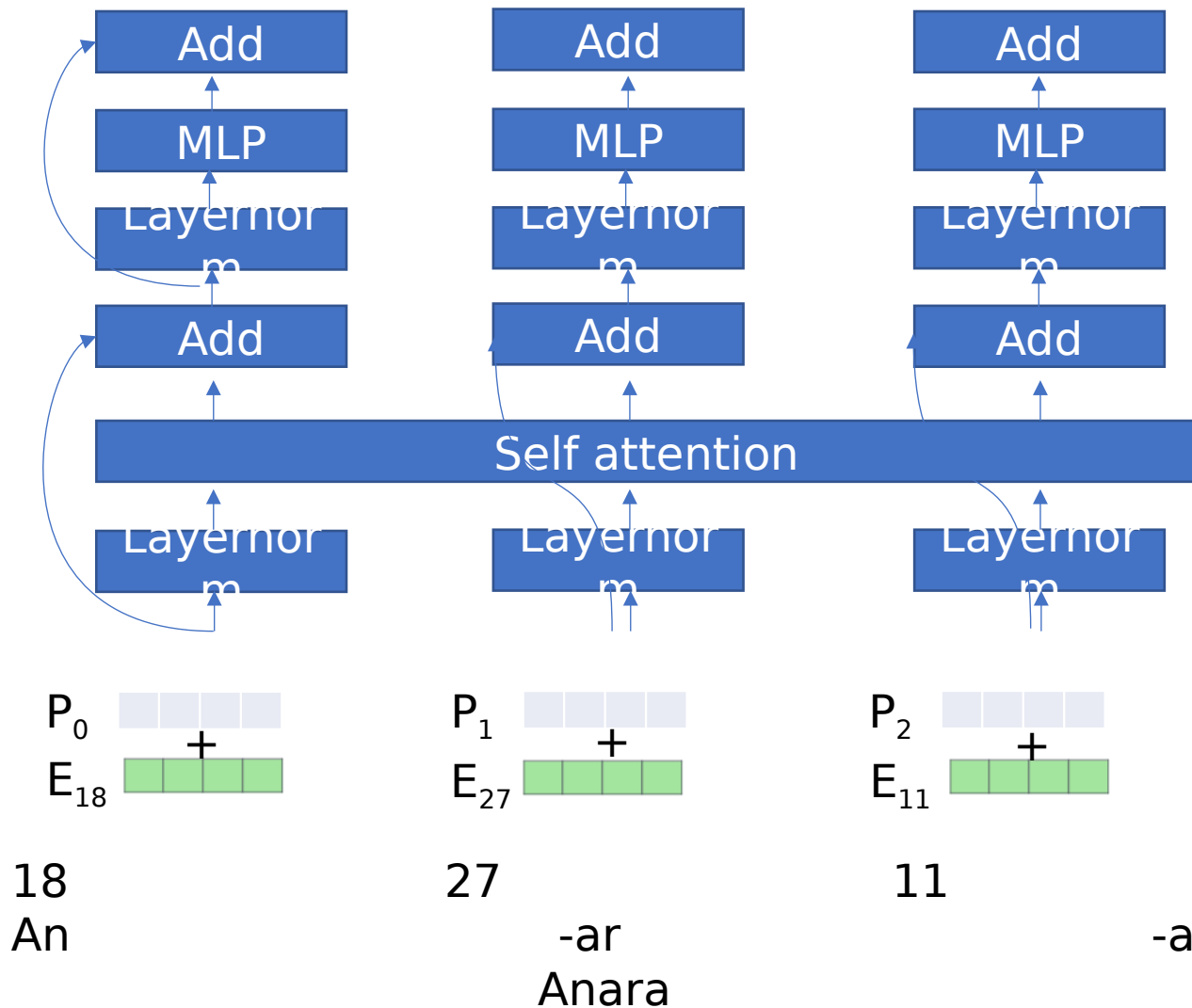
$$p(j|i) = \text{softmax}_j(q_i \cdot k_j) = \frac{e^{q_i \cdot k_j}}{\sum_{j'} e^{q_i \cdot k_{j'}}}$$

Position / order is not used

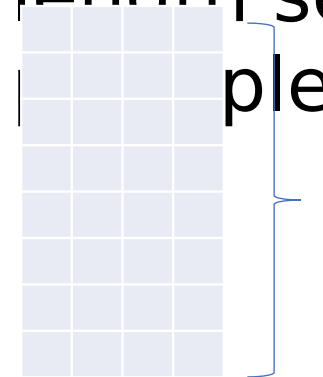
Permuting inputs would permute outputs



Position



- Most parts are processed independently, in parallel EXCEPT Self-attention
- # parameters does not grow with sequence length
- Can process arbitrary length sequences, in



Row for each token in vocabulary
For each position in the sequence

Sinusoidal Positional Encoding

Sinusoidal positional encoding is defined as follows, given the token position $i = 1, \dots, L$ and the dimension $\delta = 1, \dots, d$:

$$\text{PE}(i, \delta) = \begin{cases} \sin\left(\frac{i}{10000^{2\delta'/d}}\right) & \text{if } \delta = 2\delta' \\ \cos\left(\frac{i}{10000^{2\delta'/d}}\right) & \text{if } \delta = 2\delta' + 1 \end{cases}$$

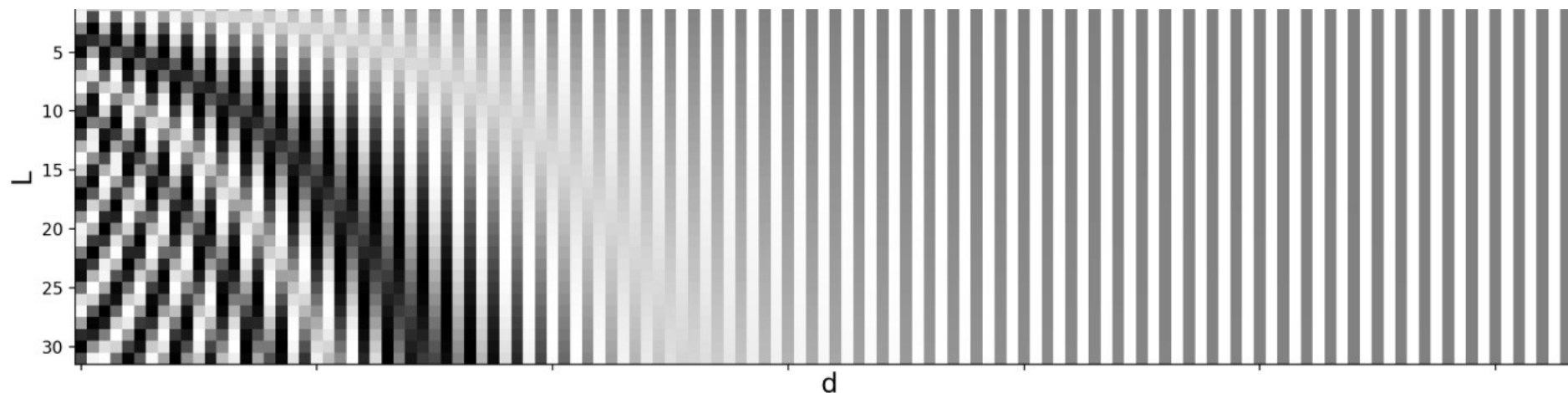
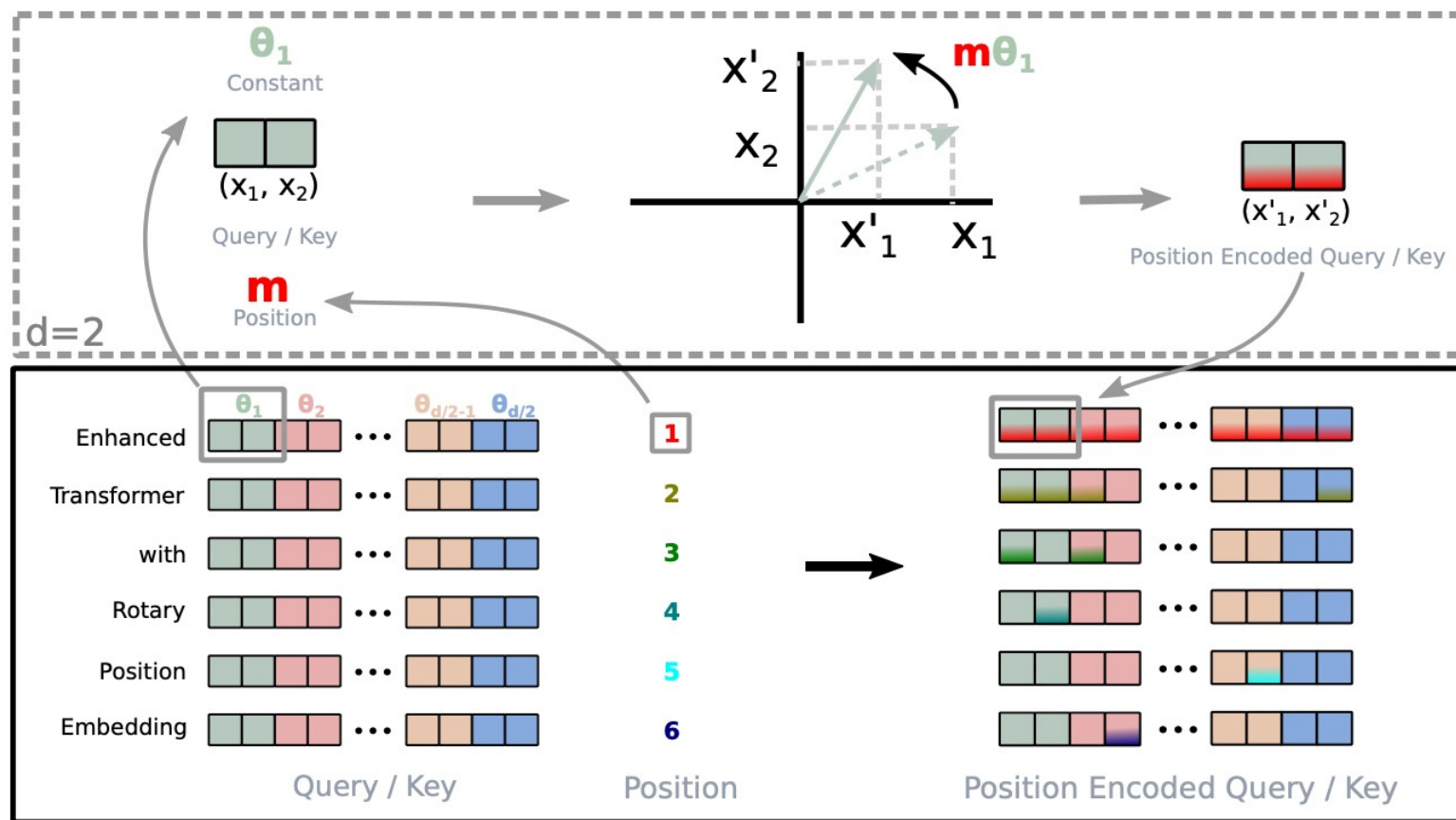


Fig. 3. Sinusoidal positional encoding with $L = 32$ and $d = 128$. The value is between -1 (black) and 1 (white) and the value 0 is in gray.

<https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/>

Newer work uses “rotation based” position embedding:

Rotation based position embedding?



Continue down the GPT
rabbit hole next time